

```

import React, { useState } from 'react';
import { Copy, Zap, Mail, Share2, Lightbulb, Sparkles, CheckCircle } from 'lucide-react';
import Head from 'next/head';

export default function Home() {
  const [contentType, setContentType] = useState('email');
  const [productInfo, setProductInfo] = useState('');
  const [audience, setAudience] = useState('');
  const [tone, setTone] = useState('professional');
  const [generatedContent, setGeneratedContent] = useState('');
  const [loading, setLoading] = useState(false);
  const [copied, setCopied] = useState(false);

  const contentTypes = {
    email: { name: 'Email Campaign', icon: Mail, prompt: 'Create a compelling email campaign'},
    social: { name: 'Social Media Posts', icon: Share2, prompt: 'Create 3 engaging social media posts'},
    ads: { name: 'Ad Copy', icon: Zap, prompt: 'Create Google/Facebook ad copy with a hook'},
    landing: { name: 'Landing Page', icon: Lightbulb, prompt: 'Create compelling landing page copy'},
    product: { name: 'Product Description', icon: Sparkles, prompt: 'Create a persuasive product description'}
  };

  const generateContent = async () => {
    if (!productInfo.trim() || !audience.trim()) {
      alert('Please fill in product info and target audience');
      return;
    }

    setLoading(true);
    setGeneratedContent('');

    try {
      const response = await fetch('/api/generate', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          contentType,
          productInfo,
          audience,
          tone,
        })
      });

      const data = await response.json();
      setGeneratedContent(data.content || 'Error generating content');
    } catch (error) {
      console.error('Error generating content:', error);
    }
  };
}

```

```

    } catch (error) {
      setGeneratedContent(`Error: ${error.message}`);
    } finally {
      setLoading(false);
    }
  };

const copyToClipboard = () => {
  navigator.clipboard.writeText(generatedContent);
  setCopied(true);
  setTimeout(() => setCopied(false), 2000);
};

return (
  <>
    <Head>
      <title>ContentAI - AI Marketing Copy Generator</title>
      <meta name="description" content="Generate high-converting marketing copy" />
      <meta name="viewport" content="width=device-width, initial-scale=1" />
    </Head>

    <div className="min-h-screen bg-gradient-to-br from-slate-900 via-slate-800" />
    <div className="border-b border-slate-700 bg-slate-900/50 backdrop-blur">
      <div className="max-w-6xl mx-auto px-6 py-8">
        <div className="flex items-center gap-3 mb-2">
          <Zap className="w-8 h-8 text-blue-400" />
          <h1 className="text-4xl font-bold text-white">ContentAI</h1>
        </div>
        <p className="text-slate-400 text-lg">Generate high-converting market
      </p>
    </div>

    <div className="max-w-6xl mx-auto px-6 py-12">
      <div className="grid lg:grid-cols-3 gap-8">
        <div className="lg:col-span-1">
          <div className="bg-slate-800 rounded-xl border border-slate-700 p-8">
            <h2 className="text-xl font-bold text-white mb-6">Create Content<

            <div className="mb-8">
              <div className="text-sm font-semibold text-slate-300 mb-2">Content Type Selection
              <div className="space-y-2">
                {Object.entries(contentTypes).map(([key, value]) => (
                  <button
                    key={key}

```

```

        onClick={() => setContentType(key)}
        className={`w-full flex items-center gap-3 px-4 py-3 rounded
        contentType === key
          ? 'bg-blue-600 text-white'
          : 'bg-slate-700 text-slate-300 hover:bg-slate-600'
        }`}
      >
        <value.icon className="w-5 h-5" />
        <span>{value.name}</span>
      </button>
    )})
  </div>
</div>

```

```

{ /* Product Info */}
<div className="mb-6">
  <label className="block text-sm font-semibold text-slate-300 mb-2">
    Product Info
  </label>
  <textarea
    value={productInfo}
    onChange={(e) => setProductInfo(e.target.value)}
    placeholder="e.g., 'Healthy protein shakes', 'SEO services for small businesses'"
    className="w-full bg-slate-700 border border-slate-600 text-slate-300"
    rows="3"
  />
</div>

```

```

{ /* Target Audience */}
<div className="mb-6">
  <label className="block text-sm font-semibold text-slate-300 mb-2">
    Target Audience
  </label>
  <textarea
    value={audience}
    onChange={(e) => setAudience(e.target.value)}
    placeholder="e.g., 'Busy entrepreneurs', 'Health-conscious workers'"
    className="w-full bg-slate-700 border border-slate-600 text-slate-300"
    rows="3"
  />
</div>

```

```

{ /* Tone */}
<div className="mb-8">
  <label className="block text-sm font-semibold text-slate-300 mb-2">
    Tone
  </label>
  <select
    value={tone}
    onChange={(e) => setTone(e.target.value)}
    className="w-full bg-slate-700 border border-slate-600 text-slate-300"
  >
    <option value="professional">Professional</option>
  </select>
</div>

```

```

        <option value="friendly">Friendly & Casual</option>
        <option value="urgent">Urgent & Compelling</option>
        <option value="creative">Creative & Fun</option>
        <option value="educational">Educational</option>
    </select>
</div>

{/* Generate Button */}
<button
    onClick={generateContent}
    disabled={loading}
    className="w-full bg-blue-600 hover:bg-blue-700 disabled:bg-sla
>
    {loading ? (
        <>
            <div className="animate-spin rounded-full h-5 w-5 border-2
                Generating...
            </div>
        </>
    ) : (
        <>
            <Sparkles className="w-5 h-5" />
            Generate Content
        </>
    )}
</button>
</div>
</div>

{/* Right Panel - Output */}
<div className="lg:col-span-2">
    <div className="bg-slate-800 rounded-xl border border-slate-700 p-8
    <div className="flex items-center justify-between mb-6">
        <h2 className="text-xl font-bold text-white">Your Content</h2>
        {generatedContent && (
            <button
                onClick={copyToClipboard}
                className="flex items-center gap-2 px-4 py-2 bg-slate-700 h
            >
                {copied ? (
                    <>
                        <CheckCircle className="w-4 h-4 text-green-400" />
                        Copied!
                    </>
                ) : (
                    <>
                        <Copy className="w-4 h-4" />
                        Copy
                    </>
                )}
            </div>
        )}
    </div>
    <div className="flex-grow-1">
        {generatedContent}
    </div>
    </div>

```

```

        </>
      )}
    </button>
  )}
</div>

{generatedContent ? (
  <div className="bg-slate-700/50 rounded-lg p-6 border border-slate-700" >
    <div className="text-slate-100 whitespace-pre-wrap text-sm leading-relaxed">
      {generatedContent}
    </div>
  </div>
) : (
  <div className="bg-slate-700/50 rounded-lg p-12 border border-slate-700" >
    <Sparkles className="w-12 h-12 text-slate-500 mb-4" />
    <p className="text-slate-400 mb-2">No content yet</p>
    <p className="text-slate-500 text-sm">Fill in your details and we'll generate content for you</p>
  </div>
)}
</div>

{/* Features Grid */}
<div className="grid grid-cols-3 gap-4 mt-8">
  {[
    { icon: Zap, label: 'Lightning Fast', value: '<5 seconds' },
    { icon: CheckCircle, label: 'High Converting', value: 'Data-driven results' },
    { icon: Sparkles, label: 'AI Powered', value: 'Claude 4' },
  ]}.map((feature, idx) => (
    <div key={idx} className="bg-slate-800 rounded-lg border border-slate-700 p-6">
      <feature.icon className="w-6 h-6 text-blue-400 mx-auto mb-2" />
      <p className="text-white font-semibold text-sm">{feature.label}</p>
      <p className="text-slate-400 text-xs">{feature.value}</p>
    </div>
  ))}
</div>
</div>
</div>

{/* Footer */}
<div className="mt-16 border-t border-slate-700 pt-8 text-center">
  <p className="text-slate-400 text-sm">
    ContentAI • Unlimited content generation • No credit card required
  </p>
</div>
</div>
</div>
</>

```

